

# La classe ReentrantLock

**Fabrice.Kordon@lip6.fr**





# En guise d'introduction

2

## Les moniteurs comme mécanismes synchronisation

-  Objet «théorique» défini par Hoare en 1974

## Première implémentation en Java

-  Intégrée à la notion d'objet
-  Forte contrainte: une seule condition par objet
-  Une seule condition (implicite)
  - Utiliser plusieurs objets? Peu souple

## Autre implémentation

-  Plus complète



# Motivation pour ReentrantLock

3

## Il y a des limites...

- 🔊 Directive «synchronized» et verrous associés aux objets
- 🔊 Quid si besoin de plusieurs conditions au sens de Hoare?
  - ▶ Une instance d'Object par «file d'attente» de processus?
  - ▶ Mais soucis pour la manipulation des données communes...  
... car on est synchronisé sur une condition ou sur l'autre...
  - ▶ Utiliser un troisième objet?
  - ▶ Exemple du Sas

## Besoin de haut niveau

- 🔊 Manipuler plusieurs conditions liées à un système de synchronisation unique
- 🔊 Possible en Java (mécanismes de haut niveau)
  - ▶ `java.util.concurrent`



# Java implémente des «verrous»

4

## Interface d'origine : Lock

 Dans `java.util.concurrent.locks`

## Méthodes

 `void lock()`

▶ Acquérir le verrou

 `Boolean tryLock()`

▶ Acquérir le verrou mais uniquement s'il est libre

 `Boolean tryLock(long d, TimeUnit u)`

▶ Acquérir le verrou avec temporisation de d unités de temps u

 `void unlock()`

▶ Libérer le verrou

 `Condition newCondition()`

▶ Créer une nouvelle condition (file de processus)

**TimeUnit (type énuméré)**

DAYS, HOURS, MICROSECONDS,  
MINUTES, SECONDS, MILLISECONDS,  
NANOSECONDS



# Manipulation des conditions

5

## Principales méthodes

 void await()

▶ Suspension de la thread jusqu'à un signal ou une interruption

 void signal()

▶ Réveille une thread suspendue sur la condition

 void signalAll()

▶ Réveille toutes les threads suspendues sur la condition

 boolean await (long d, TimeUnit u)

▶ Attente avec temporisation

 void awaitUninterruptibly()

▶ Attente (protection contre les interruptions)

 boolean awaitUntil(Date d)

▶ Attente jusqu'à une date donnée

## Une Date?

Construite avec un constructeur  
Date (avec ou sans paramètres)

**RTFM!**





# La classe ReentrantLock

6

## Constructeurs

-  `ReentrantLock()`
-  `ReentrantLock(boolean fair)`



- ▶ Possibilité de spécifier de demander l'équité dans l'ordonnancement
- ▶ Favoriser ceux qui attendent depuis longtemps
- ▶ Attention, `tryLock()` ne s'en préoccupe pas

## Quelques méthodes complémentaires

-  `int getHoldCount()`
  - ▶ Nombre de threads bloquées
-  `protected Thread getOwner()`
-  `int getQueueLength()`
  - ▶ Nombre de thread en attente de l'acquisition du verrou
-  `isFair()`
-  `isLocked()`
-  Associés aux conditions...
  - ▶ `int getWaitQueueLength(Condition c)`
  - ▶ `boolean hasWaiters(Condition c)`





# Comparaison avec synchronized & moniteurs d'objets?

7

## Les équivalences

-  synchronized (obj)
  - ▶ lock/unlock
  - ▶ Ne pas oublier de déverrouiller...
-  wait
  - ▶ await
-  notify/notifyAll
  - ▶ signal/signalAll

## Mais...

-  Possibilité de partager des conditions au sens de Hoare!
  - ▶ Sans faire de synchronized imbriqués



### Important!

Possible pour une même thread  
d'exécuter le lock() plusieurs fois  
(réentrance)

### Mais...

Toujours un seul fil actif...

